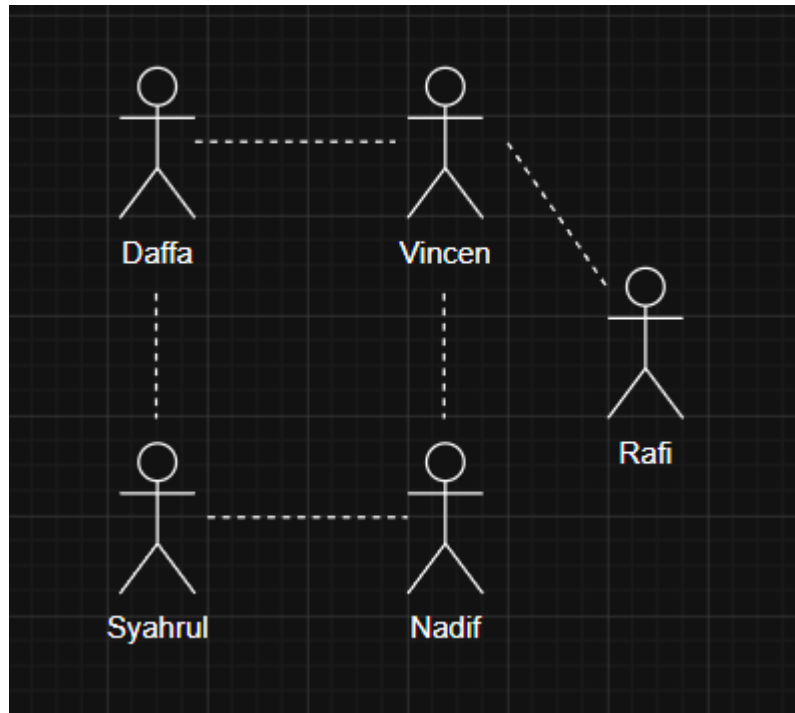


NIM : J0403251042

Nama : Faris Daffa Al Haq

1. Nodenya itu user
2. Hubungan antar node adalah kolaborasi postingan
- 3.



4. Implementasi dalam Python

A. Adjacency List

node berfungsi sebagai titik yang mewakili sebuah objek tunggal, seperti akun individu pengguna Instagram (misalnya Andi, Budi, atau Citra), sedangkan edge adalah garis penghubung antar node yang merepresentasikan bentuk hubungan atau interaksi nyata di antara entitas tersebut, yang dalam studi kasus ini adalah rekam jejak aktivitas kolaborasi postingan collab antara dua akun pengguna.

```
1 # Praktikum 4 - Adjacency List
2 # Nama: Faris Daffa Al Haq
3 # NIM: 30403251042
4
5 Windsurf: Refactor | Explain | Generate Docstring | X
6 def createGraph(V, edges):
7     adj = [[] for _ in range(V)]
8
9     # Add each edge to the adjacency list
10    for it in edges:
11        u = it[0]
12        v = it[1]
13        adj[u].append(v)
14
15        # since the graph is undirected
16        adj[v].append(u)
17    return adj
18
19 if __name__ == "__main__":
20     # Total node (Andi, Budi, Citra, Denis, Eka)
21     V = 5
22
23     # Pemetaan indeks ke nama untuk mempermudah pembacaan output nanti
24     names = ["Andi", "Budi", "Citra", "Denis", "Eka"]
25
26     # List of edges (u, v) berdasarkan relasi kolaborasi
27     edges = [
28         [0, 1], # Andi collab dengan Budi
29         [0, 2], # Andi collab dengan Citra
30         [1, 2], # Budi collab dengan Citra
31         [1, 3], # Budi collab dengan Denis
32         [2, 4], # Citra collab dengan Eka
33         [3, 4], # Denis collab dengan Eka
34     ]
35
36     # Build the graph using edges
37     adj = createGraph(V, edges)
38
39     print("Adjacency List Representation:")
40     for i in range(V):
41
42         # Print the vertex (dengan tambahan nama agar lebih jelas)
43         print(f"({i}):", end=" ")
44
45         for j in adj[i]:
46
47             # Print its adjacent (menampilkan nama tetangganya)
48             print(names[j], end=" ")
49         print()
```

```
Eka (4): Citra, Denis,
PS C:\IPB\SMT2\ALGORITMA&STRUKTUR_DATA\30403251042_FarisDaffaAlHaq_A1\Pertemuan11> &
Adjacency List Representation:
Andi (0): Budi, Citra,
Budi (1): Andi, Citra, Denis,
Citra (2): Andi, Budi, Eka,
Denis (3): Budi, Eka,
Eka (4): Citra, Denis,
PS C:\IPB\SMT2\ALGORITMA&STRUKTUR_DATA\30403251042_FarisDaffaAlHaq_A1\Pertemuan11>
```

B. Adjacency Matrix

Node diwakili oleh baris dan kolom matriks yang merujuk pada entitas individu pengguna Instagram (seperti Andi, Budi, Citra, Denis, dan Eka), sedangkan edge direpresentasikan secara spesifik oleh angka 1 pada titik persimpangan antara baris dan kolom tertentu, yang secara logis membuktikan adanya ikatan hubungan atau riwayat kolaborasi postingan kolaborasi antara dua entitas tersebut (sementara angka 0 menandakan ketiadaan edge atau belum pernah berkolaborasi).

```

1 # Praktikum 4 - adjacency matrix
2 # Nama: Faris Daffa Al Haq
3 # NIM: J0403251042
4
5 Windsurf: Refactor | Explain | Generate Docstring | X
6 def createGraph(V, edges):
7     mat = [[0 for _ in range(V)] for _ in range(V)]
8
9     # Add each edge to the adjacency matrix
10    for it in edges:
11        u = it[0]
12        v = it[1]
13        mat[u][v] = 1
14
15    # since the graph is undirected
16    mat[v][u] = 1
17    return mat
18
19 if __name__ == "__main__":
20     # Total node (Andi, Budi, Citra, Denis, Eka)
21     V = 5
22
23     # Pemetaan indeks ke nama
24     names = ["Andi", "Budi", "Citra", "Denis", "Eka"]
25
26     # List of edges (u, v) berdasarkan relasi kolaborasi
27     # 0=Andi, 1=Budi, 2=Citra, 3=Denis, 4=Eka
28     edges = [
29         [0, 1], # Andi collab dengan Budi
30         [0, 2], # Andi collab dengan Citra
31         [1, 2], # Budi collab dengan Citra
32         [1, 3], # Budi collab dengan Denis
33         [2, 4], # Citra collab dengan Eka
34         [3, 4], # Denis collab dengan Eka
35     ]
36
37     # Build the graph using edges
38     mat = createGraph(V, edges)
39
40     print("Adjacency Matrix Representation:")
41
42     # Menampilkan header nama agar matriks mudah dibaca
43     print(" " + " ".join(names))
44
45     for i in range(V):
46         # Menampilkan nama baris di sebelah kiri
47         print(f"{names[i]:<6}: ", end=" ")
48
49         for j in range(V):
50             # Format jarak agar angka sejajar dengan nama di header
51             print(f"{mat[i][j]:<5}", end=" ")
52         print()

```

```

PS C:\IPB\SMT2\ALGORITMA&STRUKTUR_DATA\J0403251042_FarisDaffaAlHaq_A1\Pertemuan11> & "
Adjacency Matrix Representation:
      Andi Budi Citra Denis Eka
Andi : 0   1   1   0   0
Budi : 1   0   1   1   0
Citra: 1   1   0   0   1
Denis: 0   1   0   0   1
Eka  : 0   0   1   1   0
PS C:\IPB\SMT2\ALGORITMA&STRUKTUR_DATA\J0403251042_FarisDaffaAlHaq_A1\Pertemuan11>

```

5. Analisis Singkat

- A. Graph ini termasuk ke dalam jenis undirected graph (graph tidak berarah). Alasannya adalah fitur kolaborasi di Instagram bersifat mutlak dan dua arah (simetris). Jika Andi mengundang Budi untuk kolaborasi postingan dan Budi menerima, maka postingan tersebut akan muncul di feed Andi maupun feed Budi secara bersamaan.
- B. Karena jika kedua akun tidak terhubung oleh edge, artinya mereka belum pernah membuat postingan kolaborasi bersama, meskipun mungkin saling follow.
- C. Adjacency List jauh lebih cocok dibandingkan Adjacency Matrix, Jika menggunakan matriks, kita harus membuat tabel yang sangat banyak yang mayoritas isinya adalah angka. Adjacency List, mencatat siapa yang benar-benar pernah berkolaborasi
- D. Graph jejaring sosial ini termasuk sparse graph karena perbandingan matematis antara jumlah hubungan yang terjadi dengan jumlah hubungan maksimal yang mungkin terjadi.
- E. Perbandingan antara Adjacency List dan Adjacency Matrix terlihat pada antara efisiensi memori dan kecepatan pencarian data. Adjacency Matrix unggul dalam kecepatan akses karena mampu mengecek keberadaan relasi antar dua node secara instan.

6. Kesimpulan Singkat

Graph merupakan struktur data yang sangat efektif untuk merepresentasikan hubungan antar objek, seperti kolaborasi antar akun di media sosial. Node bertindak sebagai representasi pengguna, sementara Edge merepresentasikan interaksi atau hubungan (kolaborasi postingan). Dalam implementasinya, pemilihan metode representasi sangat bergantung pada karakteristik data. Adjacency List menjadi pilihan paling efisien untuk kasus media sosial yang bersifat sparse karena menghemat penggunaan memori, sedangkan Adjacency Matrix lebih unggul dalam kecepatan akses pencarian hubungan langsung meskipun memerlukan ruang penyimpanan yang lebih besar.